

Running Kaltura API Scripts to Build Accessibility Reports

Overview

Kaltura reports for a specified timeframe can be generated through a single Python script.

The script uses `media.list` as the foundation of the report, automatically divides large date ranges into smaller reporting windows when necessary, retrieves all matching media records, and then collects related caption and category information.

The script then:

- combines the results using each media entry's `entryId`
- summarizes caption, transcript, and category data
- creates detailed and leadership-facing CSV reports
- writes raw supporting CSV files and a run log

You do not need to manually create several date-based CSV reports and combine them with another script.

1. Download and Update the Python Script

Use the shareable Python script (located in the Git repository):

None

```
kaltura_media_caption_category_report_complete_shareable.py
```

The script does **not** contain a real Partner ID, Category ID, KS, Admin Secret, or institutional credentials.

Near the top of the script, look for:

None

```
# =====  
# UPDATE THESE VALUES BEFORE RUNNING A NEW REPORT  
# =====
```

Update these four values before each report:

None

```
# 1) Partner ID
PARTNER_ID = 1234567 # <-- UPDATE THIS

# 2) Root / ancestor category ID
CATEGORY_ANCESTOR_ID = "123456789" # <-- UPDATE THIS

# 3) Report start and end dates as UNIX timestamps
REPORT_START_TIMESTAMP = 1704067200 # <-- UPDATE THIS
REPORT_END_TIMESTAMP = 1706745599 # <-- UPDATE THIS

# 4) Report label
REPORT_LABEL = "2024Jan1toJan31" # <-- UPDATE THIS
```

Do **not** paste your KS or Admin Secret into the Python file.

2. Find Your Category ID

The report uses `categoryAncestorIdIn` to limit results to media under a specific Kaltura category hierarchy.

If you use Kaltura with Canvas or another LMS:

1. Identify the root category used by your LMS/KAF integration. For example, the root category may be named `Canvas`.
2. Sign in to KMC.
3. Go to **Content > Categories**.
4. Search for the root category name.
5. Find the matching category.
6. Copy the number shown in the **ID** column.

Example:

None

```
Category name: Canvas
Category ID: 123456789
```

Enter that number in the Python script:

None

```
CATEGORY_ANCESTOR_ID = "123456789"
```

The script uses it here:

None

```
"categoryAncestorIdIn": CATEGORY_ANCESTOR_ID
```

3. Access the Kaltura API and Generate a KS

Before running a report, you need:

- Partner ID
- Admin Secret
- Kaltura user ID

Go to:

<https://developer.kaltura.com/api-docs/Overview>

Sign in using your Kaltura credentials and search for:

None

```
session.start
```

Enter:

None

```
secret = Admin Secret  
partnerId = Partner ID  
type = Admin [2]  
userId = your Kaltura user ID
```

Run the request.

The result is a Kaltura Session, or **KS**, string.

A KS expires, so if the report later returns an invalid-session error, generate a new KS.

Keep the Admin Secret and KS private.

Copy the KS string. You will need the KS string in step 5, below.

4. Create a Folder for the Script and Reports

Create a folder named:

```
None  
KalturaReports
```

inside your **Documents** folder.

Save the Python script in that folder.

On a Mac, the folder path will be:

```
None  
~/Documents/KalturaReports
```

This keeps the Python script, CSV reports, and run log together in one place.

5. Set the KS in Terminal

On a Mac, open **Terminal**.

Go to the folder containing the Python script. For example:

```
None  
cd ~/Documents/KalturaReports
```

Set the current KS:

None

```
export KALTURA_KS='PASTE_YOUR_CURRENT_KS_HERE'
```

Do not paste the KS into Terminal by itself.

You can confirm a value is stored without displaying the KS:

None

```
echo ${#KALTURA_KS}
```

If the result is greater than 0, the KS is stored in the current Terminal session.

6. Choose the Report Dates

The Python script uses UNIX timestamps for the start and end dates.

Update:

None

```
REPORT_START_TIMESTAMP = ...  
REPORT_END_TIMESTAMP = ...
```

Use timestamps that match the timezone and full date range you want to report on. You can enter dates in your AI of choice and ask for the UNIX timestamp.

Also update:

None

```
REPORT_LABEL = "..."
```

The report label is used in the output filenames.

7. Check the Python Script

Before running a long report, check the Python syntax:

```
None  
python3 -m py_compile  
kaltura_media_caption_category_report_complete_shareable.py
```

If Terminal returns to the prompt without an error, continue.

If you run into an error, check that you are using the correct python file name.

8. Run the Report

Run:

```
None  
python3 kaltura_media_caption_category_report_complete_shareable.py
```

Keep Terminal open while the report runs.

When the script finishes successfully, Terminal displays:

```
None  
Done.
```

9. How media.list Is Used

`media.list` is the base report because it returns the main media records for the selected timeframe and category scope.

The script builds a filter similar to:

```
None  
{  
  "filter": {  
    "objectType": "KalturaMediaEntryFilter",
```

```
    "categoryAncestorIdIn": "YOUR_CATEGORY_ID",
    "createdAtGreaterThanOrEqual": 1704067200,
    "createdAtLessThanOrEqual": 1706745599,
    "orderBy": "+createdAt"
  },
  "pager": {
    "pageSize": 500,
    "pageIndex": 1
  }
}
```

`media.list` provides the media entry IDs used to retrieve the caption and category information.

10. How the Script Handles the 10,000-Entry Limit

Kaltura list queries have an effective 10,000-result ceiling for a single large query.

The script uses:

None

```
MAX_RESULTS_PER_WINDOW = 9000
```

If the requested timeframe contains more than the safe threshold, the script automatically divides the timeframe into smaller date windows.

It then:

1. retrieves each safe window
2. combines the results
3. removes duplicate media IDs
4. continues with caption and category reporting

You do not need to manually create separate monthly or quarterly reports.

11. Caption and Transcript Data

The script runs `captionAsset.list` for the media IDs returned by `media.list`.

The report can include:

- caption asset count
- caption availability
- caption languages
- default caption labels
- caption accuracy values
- accessibility asset types
- transcript availability
- associated transcript IDs

The field:

None

`transcript_available`

shows **Yes** when Kaltura returns one or more values in `associatedTranscriptIds`.

This can be especially useful for audio-only media.

12. Category Data

The script runs:

None

`categoryEntry.list`

to identify the categories assigned to each media entry.

It then runs:

None

`category.list`

to retrieve readable category names and full paths.

13. Output Files

The script creates:

None

```
KalturaMediaRaw_[REPORT_LABEL].csv  
KalturaCaptionRaw_[REPORT_LABEL].csv  
KalturaCategoryEntryRaw_[REPORT_LABEL].csv  
KalturaCategoryRaw_[REPORT_LABEL].csv  
KalturaMediaCaptionCategoryFinal_[REPORT_LABEL].csv  
KalturaMediaCaptionCategoryLeadership_[REPORT_LABEL].csv  
KalturaMediaCaptionCategoryRunLog_[REPORT_LABEL].txt
```

The final CSV contains the detailed report.

The leadership CSV contains a smaller set of selected fields.

The run log records the report settings, safe media windows, record totals, and output filenames.

14. Read-Only Nature of the Script

The reporting script uses Kaltura list operations:

None

```
media.list  
captionAsset.list  
categoryEntry.list  
category.list
```

It does not delete or modify Kaltura media, captions, or categories.

It only reads Kaltura data and writes CSV and log files to the local computer.